

بروتوكول HTTP: الجانب التطبيقي

HTTP Protocol: Practical Part

إعداد: م. ربيع عبد الحفيظ الشويهي
Prepared by: Eng. Rabei Abdulhafid Shwehdi



حقوق الملكية الفكرية وحماية المؤلف

"جميع حقوق الملكية الفكرية لهذا العمل محفوظة للمؤلف: م. ربيع عبد الحفيظ الشويهي © 2026. إن هذا المحتوى العلمي مُعد خصيصاً للأغراض التعليمية والبحثية. يُسمح للطلبة والباحثين بتداول هذا الملف، أو تصويره، أو استخدامه في إطار الدراسة الأكاديمية بشرط الإشارة الصريحة إلى المصدر والمؤلف. يُمنع منعاً باتاً إعادة نشر هذا العمل أو جزء منه لأغراض تجارية، أو تعديله، أو نسبه لغير صاحبه، أو استغلاله في منصات مدفوعة الأجر دون إذن كتابي مسبق من المؤلف. إن الأكواد البرمجية المرفقة تخضع لرخصة المصادر المفتوحة المذكورة في مستودع GitHub الخاص بالمادة".

اقرار بالتعاون التقني (AI Collaboration Acknowledgement)

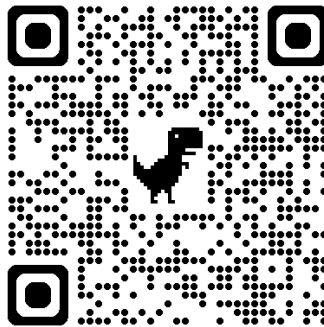
"تم تطوير وتحسين الأكواد البرمجية والخوارزميات الواردة في هذا الجزء من التجربة بالتعاون مع تقنية Gemini AI، وذلك لضمان اتباع أفضل الطرق والأساليب في البرمجة الحديثة، وتعزيز كفاءة إدارة الأخطاء في الأنظمة المدمجة (Embedded Systems). كما نود التنويه إلى أن الأشكال التوضيحية والرسوم البيانية المستخدمة في هذا الدليل قد تم توليدها وتطويرها باستخدام نفس التقنية، لتقديم محاكاة بصرية دقيقة تدعم العملية التعليمية والتدريبية".

تنويه بخصوص المصادر المفتوحة (Open Source Access)

"لضمان تحقيق أقصى استفادة تعليمية ومواكبة لأساليب التوثيق البرمجي الحديثة، نود إحاطة أبنائنا الطلبة والباحثين علماً بأن جميع الأكواد المصدرية (Source Codes) ، المخططات الهندسية، والتحديثات الدورية لهذه المادة (النظرية والعملية) متاحة بشكل مفتوح عبر الحقيبة الرقمية الخاصة بقسم التحكم الآلي على منصة **GitHub**. يمكنكم الوصول إلى كافة ملفات التجارب والمحاكاة عبر الرابط التالي:"

[اضغط هنا](#)

مسح الرمز للوصول السريع (QR Code)



يُنصح بمتابعة الرابط بانتظام للحصول على أحدث الإصدارات المصححة والمطورة من الأكواد والخوارزميات.

1. مقدمة

1.1 نظرة عامة على التجارب

في هذا الجزء، سنقوم بتطبيق المفاهيم التي درسناها في الجزء النظري الخاص ببروتوكول الاتصال HTTP حيث يهدف هذا الجانب العملي إلى تمكين الطالب من تصميم منظومة متكاملة تبدأ من استشعار البيانات وتنتهي بعرضها واحتراف التعامل معها برمجياً.

1.2 منهجية العمل

تعتمد التجارب القادمة على المنهجية التالية:

1. تصميم واجهات الويب: (Webpages) بناء صفحات تفاعلية ترسل طلبات GET Request.
2. برمجة وحدة التحكم: (Wemos D1 R1) تهيئة اللوحة لتعمل كخادم (Server) يستقبل الطلبات ويرسل استجابة (Response) محملة ببيانات الحساس متمثلة في درجة الحرارة والرطوبة داخل المعمل.
3. بيئة التطوير والمحاكاة: استخدام بيئة VS Code مع إضافة Live Server لمحاكاة خادم الصفحات، مما يوفر بيئة تسمح بمراقبة عملية ارسال الطلبات واستقبال الاستجابة مراقبة أي تغيير في التصميم بسهولة ويسر.
4. التطوير: سيتم التدرج في التصميم من النصوص البسيطة، مروراً بالأيقونات التوضيحية، وصولاً إلى استخدام Circular Progress Bar لعرض البيانات بشكل هندسي جذاب.

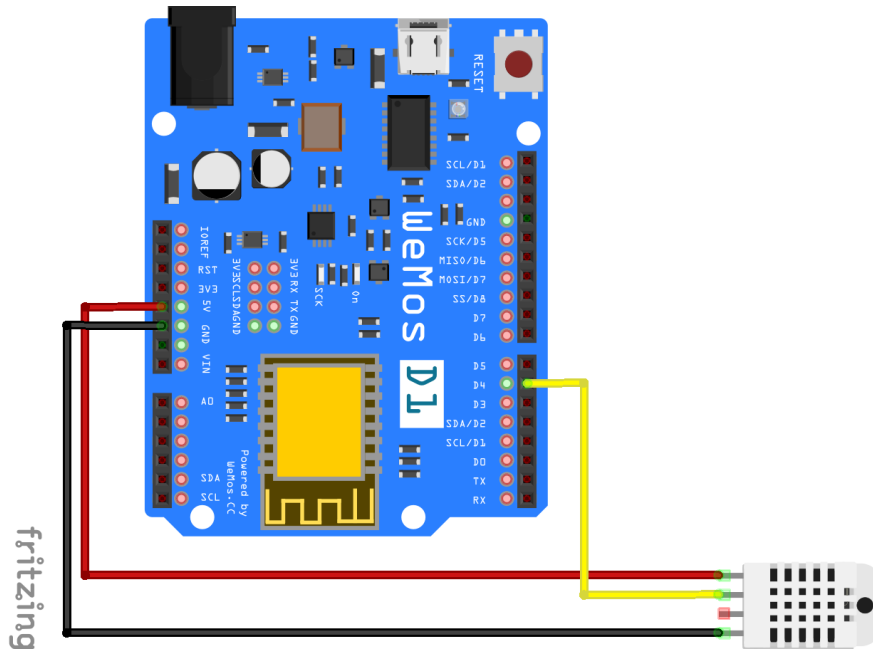
1.3 البنية التقنية للبيانات

1. لغة التواصل: سيتم الاعتماد كلياً على كود JavaScript لإدارة عملية إرسال الـ Request واستقبال الـ Response دون الحاجة لإعادة تحميل الصفحة.
2. عرض البيانات: سيتم تمرير البيانات عبر كود JavaScript الى مكونات صفحة HTML.
3. هيكلية البيانات: الرد المرسل من اللوحة سيكون بهيكلية JSON، لضمان السرعة والتوافق مع معايير الويب الحديثة.

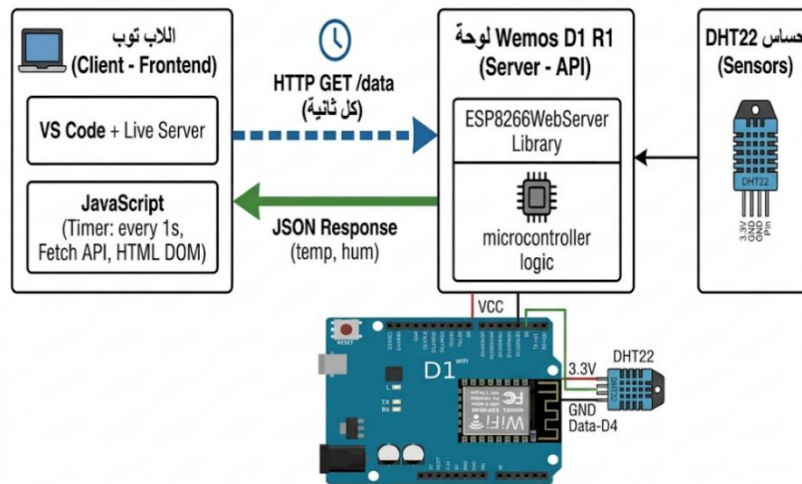
1.4 الهدف التعليمي:

1. إتقان الربط البرمجي بين العتاد (Hardware) وبرمجيات الويب (Software).
2. التعامل مع بروتوكول HTTP في بيئة الـ IoT.
3. فهم هيكلية البيانات من نوع JSON وكيفية معالجتها برمجياً.

1.5 توصيل الدائرة:



1.6 المخطط الصندوقي:



1.6.1 تحليل المخطط الصندوقي:

1. المستعرض (Client): هي واجهة المستخدم المكتوبة بـ HTML، ويقوم كود الـ JavaScript فيها عبر (Live Server) بدور "طالب البيانات". هو من يبدأ الاتصال كل ثانية.
2. لوحة (Wemos) (Server): تعمل كخادم ويب (HTTP Server) ينتظر طلبات الـ Client
3. شبكة Wi-Fi: هي "جسر الاتصال" الذي تمر عبره طلبات HTTP.
4. حساس (DHT22) (Hardware): هو "مصدر البيانات" الفعلي الذي يقوم بقياس الحرارة والرطوبة.

2. التجارب

2.1 التجربة 1 (الجزء الاول)

في هذه التجربة سنقوم بتصميم صفحة HTML بسيطة ترسل طلبا باستخدام GET كل ثانية الى لوحة WEMOS والتي بدورها تقوم بالرد بإرسال قراءة الحساس DHT22 المتصلة بها (حرارة ورطوبة) الى الصفحة ليتم عرض هذه القراءات على الصفحة بمعدل تحديث مرة كل ثانية.

2.1.1 تصميم الصفحة

أولا الكود البرمجي

```
<!doctype html>
<html>
  <body>
    <div>
      <span class="label-text">Temp = </span>
      <span id="temp">Loading...</span>
      <span class="unit">°C</span>
    </div>
    <div>
      <span class="label-text">Humi = </span>
      <span id="humi">Loading...</span>
      <span class="unit"> %</span>
    </div>

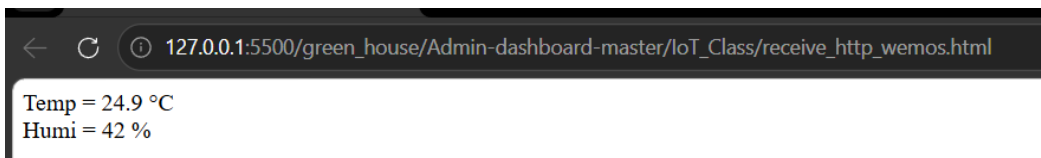
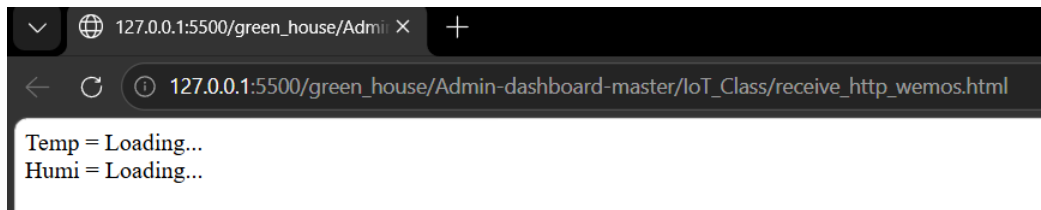
    <script>
      // جلب القراءات من اللوحة دون تجميد الواجهة (async) تعريف دالة غير متزامنة
      async function loadTemp() {
        // انتظار استلام الرد Wemos لوحة IP إلى HTTP GET إرسال طلب
        const response = await fetch("http://192.168.1.13");
        // تحويل نص الرد المستلم إلى كائن برمجي بتنسيق JSON
        const data = await response.json();
        // تحديث محتوى عنصر الحرارة في الصفحة بالقيمة القادمة من الحساس
        document.getElementById("temp").textContent = data.temp;
        // تحديث محتوى عنصر الرطوبة في الصفحة بالقيمة القادمة من الحساس
        document.getElementById("humi").textContent = data.hum;
        // المتصفح لأغراض المراقبة والتصحيح Console طباعة البيانات المستلمة في
        console.log("Data updated: ", data);
      }
      // استدعاء الدالة لمرة واحدة فور فتح الصفحة لعرض البيانات مباشرة
      loadTemp();
      // ضبط مؤقت زمني لتكرار تنفيذ الدالة تلقائياً كل 1000 ملي ثانية (ثانية واحدة)
      setInterval(loadTemp, 1000);
    </script>
  </body>
</html>
```

✓ عنصر `<span>`: تم استخدامه هنا لأنه عنصر "سطري" (inline) يسمح بوضع النص والوحدة والقيمة بجانب بعضهم البعض بشكل متناسق.

✓ دالة `fetch`: هي المسؤولة عن التواصل مع لوحة Wemos D1 R1 عبر الشبكة حيث تقوم بإرسال الطلب request (<http://192.168.1.13>) وتنتظر الرد.

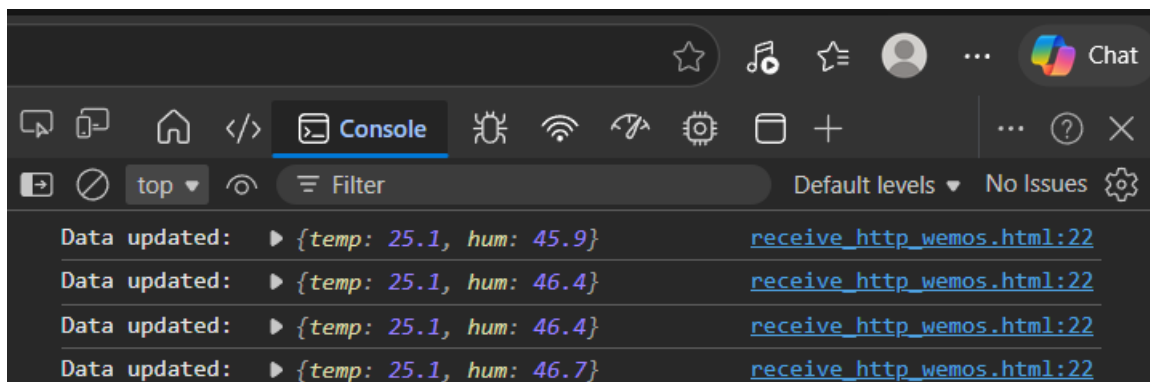
✓ دالة `setInterval`: هي المسؤولة عن جعل الصفحة "حية" ومحدثة باستمرار بمعدل ثانية واحدة دون تدخل المستخدم ففي كل ثانية يتم استدعاء الدالة `loadTemp()` وهذه الدالة هي التي تحتوى على كود ارسال الطلب.

كما يظهر في الشكل بالأسفل الصفحة تصميم بسيط عبارة عن عنوان تليه قراءة الحساس المرسله من لوحة WEMOS عبر بروتوكول HTTP قبل ان يبدأ الاتصال واستقبال البيانات تعرض ما يفيد بانتظار تحميل البيانات وفور نجاح الاتصال تبدأ القراءات بالظهور وتحديث كل ثانية.



بالإمكان متابعة حالة الاتصال بالضغط على زر F12 الدخول على نافذة DEBUG كما هو موضح بالأسفل

حيث يظهر عنوان اللوحة IP ADDRESS والقيم المستلمة (temp:25.1 hum:45.9)



2.1.2 ملاحظات مهمة:

1. نلاحظ في السطرين التاليين انه تم إعطاء اسم مميز (id) الى وسم (HTML TAG) كما هو واضح في السطر التالي

```
<span id="temp">Loading...</span>
```

ثم تم استخدام هذا id في كود js لتمرير قيمة درجة الحرارة المستلمة الى الوسم لتعرض على الشاشة كما بالسطر التالي

```
document.getElementById("temp").textContent = data.temp;
```

2. لطباعة البيانات المستلمة على console:

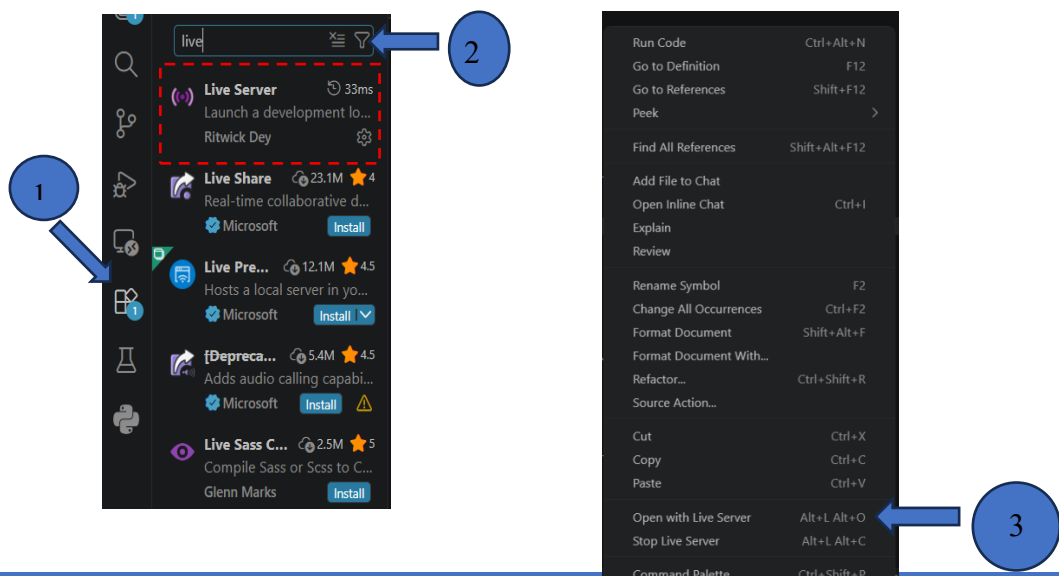
```
console.log("Data updated: ", data);
```

هنا البيانات بصيغة JSON مخزنة داخل المتغير data :

- للوصول لدرجة الحرارة data.temp
- للوصول الى نسبة الرطوبة data.hum

3. لإضافة live server

- اضغط على زر extension
- اكتب في محرك البحث live server اختر الخيار الظاهر في الصورة واضغط على زر التثبيت
- لتشغيل live server اضغط بزر الفارة الأيمن واختر open with live server



2.1.3 شرح كود Wemos

1. الشكل التالي يوضح جزء استدعاء المكتبات والتعريفات

```

1 //=====
2 //ESP8266 Web Server Peripheral Data
3 //=====
4 #include <ESP8266WiFi.h> // مكتبة التحكم في شريحة الواي فاي لـ ESP8266
5 #include <WiFiClient.h> // مكتبة تتيح للوحة العمل كعميل للاتصال بالشبكات
6 #include <ESP8266WebServer.h> // مكتبة لإنشاء خادم ويب (Web Server)
7 #include <DHT.h> // مكتبة التعامل مع حساسات الحرارة والرطوبة من نوع DHT
8 #include <ArduinoJson.h> // مكتبة لتحويل البيانات إلى تنسيق JSON ليفهمها الويب

10 DHT dht(2, DHT22); // على المنفذ رقم 2 DHT22 تعريف الحساس
11 ESP8266WebServer server(80); // 80 إنشاء خادم ويب يعمل على المنفذ القياسي رقم
12 //-----

14 // تعريف اسم شبكة الواي فاي وكلمة المرور للاتصال بها
15 const char* ssid = "*****";
16 const char* password = "*****";
17 float HUM=0; // متغير لتخزين قيمة الرطوبة (عدد عشري)
18 float TEMP=0; // متغير لتخزين قيمة الحرارة (عدد عشري)

20 // ثابتة بسعة 128 بايت لتجهيز البيانات قبل إرسالها JSON وثيقة
21 StaticJsonDocument<128> doc;

```

2. الشكل التالي يوضح دالة التهيئة مثل الربط مع الشبكة وتجهيز دالة الاستجابة sever.on

```

22 void setup()
23 {
24   pinMode(LED_BUILTIN,OUTPUT); // ضبط الليد المدمج في اللوحة كـ "مخرج"
25   digitalWrite(LED_BUILTIN,HIGH); // إطفاء الليد في البداية
26   Serial.begin(115200); // بدء الاتصال التسلسلي لمراقبة البيانات بسرعة 115200
27   dht.begin(); // DHT22 تشغيل حساس

29   WiFi.begin(ssid, password); // البدء في عملية الاتصال بشبكة الواي فاي
30   Serial.print("\n\rWorking to connect");
31   // حلقة انتظار لحين تمام الاتصال بالشبكة
32   while (WiFi.status() != WL_CONNECTED) {delay(500); Serial.print(".");}

33   Serial.println("\nConnected to WiFi");
34   Serial.print("IP address: ");
35   Serial.println(WiFi.localIP()); // الذي حصلت عليه اللوحة (مهم جداً للويب) IP طباعة عنوان الـ

36   // تحديد أنواع "الترويسات" التي نريد استلامها من المتصفح ومعالجتها
37   const char * headerkeys[] = {"User-Agent", "Host", "Accept-Language"} ;
38   size_t headerkeyssize = sizeof(headerkeys) / sizeof(char*);
39   server.collectHeaders(headerkeys, headerkeyssize);

40   server.on("/", send_data); // عندما يطلب أحد الرابط الأساسي (/) نفذ دالة "send_data" إخبار الخادم
41   server.begin(); // تشغيل الخادم فعلياً ليكون جاهزاً لاستقبال الطلبات
42   Serial.println("HTTP server started");
43 }

```

3. الشكل التالي يوضح الدالة المسؤولة عن التنصت على منفذ http لاستقبال اى طلب إضافة الى دالة قراءة الحساس DHT22 والتي يتم استدعاؤها في كل مرة نحتاج الى تحديث بيانات الحرارة والرطوبة

```
45 void loop() {
46   server.handleClient();           // الاستمرار في مراقبة ومعالجة طلبات العملاء القادمة للمتصفح
47 }

48 void read_sensor() {              // دالة مخصصة لقراءة البيانات من الحساس الفيزيائي
49   HUM=dht.readHumidity();          // HUM قراءة الرطوبة وحفظها في المتغير
50   TEMP=dht.readTemperature();      // TEMP قراءة الحرارة وحفظها في المتغير
51   Serial.println(HUM);             // للتأكد Serial Monitor عرض القراءات في الـ
52   Serial.println(TEMP);
53 }
```

4. الشكل التالي يوضح دالة الاستجابة والتي تستدعى بواسطة دالة server.on فتقوم باستدعاء دالة read_sensor و بعد تحديث قراءات الحرارة والرطوبة يتم عبر مكتبة JSON بصياغة القيم بهيكلية JSON ثم ارسالها بواسطة دالة sever.send كاستجابة الى طلب المتصفح.

```
54 void send_data() {                // الدالة المسؤولة عن الاستجابة لطلب المتصفح
55   Serial.println("--- New Request Received ---");

56   // حلقة تكرارية لطباعة معلومات المتصفح الطالب للبيانات (لأغراض المراقبة)
57   for (int i = 0; i < server.headers(); i++) {
58     Serial.print(server.headerName(i));
59     Serial.print(": ");
60     Serial.println(server.header(i));
61   }

62   read_sensor();                  // استدعاء دالة قراءة الحساس لتحديث القيم قبل الإرسال

63   // بالوصول للبيانات من عنوان مختلف (CORS) سطر حيوي: السماح لمفحة الويب
64   server.sendHeader("Access-Control-Allow-Origin", "*");

65   doc["temp"] = TEMP;             // تحت اسم JSON وضع قيمة الحرارة داخل هيكل
66   doc["hum"] = HUM;               // تحت اسم JSON وضع قيمة الرطوبة داخل هيكل

67   char buffer;                   // مصفوفة نصية لتخزين البيانات بعد تحويلها لنص
68   serializeJson(doc, buffer);     // ليتم إرساله عبر الشبكة (String) إلى نص JSON تحويل كائن

69   // والقيم الفعلية، JSON إرسال الرد النهائي للمتصفح: كود النجاح 200، نوع البيانات
70   server.send(200, "application/json", buffer);
71 }
```

في هذا الكود يظهر بوضوح استخدام طريقة event driven فلا يتم قراءة الحساس او الارسال الا بعد التحفيز عن طريق الحدث وهو الطلب القادم من المتصفح مما يوفر الطاقة وموارد النظام ويشغل الشبكة الا للضرورة.

توصيات هندسية لضمان استقرار النظام (Important Notes)

لضمان نجاح التجربة وتجنب الأخطاء البرمجية الشائعة عند التعامل مع لوحات Wemos D1 R1، يجب مراعاة النقاط التالية:

1. إذن الوصول (CORS Policy):

أحياناً يرفض المتصفح استلام البيانات من اللوحة لأسباب أمنية. لضمان السماح للوحة بإرسال البيانات للمتصفح دون قيود، يجب إضافة السطر التالي داخل دالة الاستجابة في كود الـ Arduino:

```
server.sendHeader("Access-Control-Allow-Origin", "*");
```

2. ديناميكية عنوان الـ IP:

يجب الأخذ في الاعتبار أن عنوان الـ IP الخاص باللوحة قد يتغير في كل مرة يتم فيها الربط مع نقطة الوصول (Access Point). لذا، يجب دائماً مراجعة العنوان الجديد عبر الـ Serial Monitor وتحديثه في كود الـ JavaScript لضمان استمرار الاتصال.

3. إدارة أخطاء الاتصال (Error Handling):

في كود الـ JavaScript، يفضل دائماً استخدام جملة try...catch أو تفقد حالة الاستجابة عبر if(response.ok). يساعد هذا في إظهار "رسالة خطأ" للمستخدم على الشاشة في حال انقطع الاتصال باللوحة، وهذا ما سيتم في التجربة الثانية.

4. تجنب إجهاد المتحكم (Server Stress):

نظراً لمحدودية موارد المتحكمات الدقيقة، يجب عدم الضغط على الخادم بطلب البيانات بسرعة عالية (مثل طلب كل 100ms). هذا الضغط قد يؤدي إلى تشنج النظام وتوقفه عن العمل؛ لذا يُنصح بأن يكون الفاصل الزمني بين الطلبات ثانية واحدة أو أكثر.

2.2 التجربة 1 (الجزء الثاني)

في هذا الجزء سنقوم بما يلي:

تطوير تصميم الصفحة بإضافة (Style Rule) الخاصة بـ CSS (Cascading Style Sheets) لإضفاء بعض الجمالية والاحترافية لشكل الصفحة دون ان يكون هناك تغيير في هيكلية التصميم ف (Style Rule) تؤثر على شكل التصميم من ناحية الألوان والتنسيق وتقسيم الشاشة فقط.

2.2.1 مقدمة في لغة CSS (Cascading Style Sheets)

إذا كانت لغة HTML هي الهيكل العظمي للموقع (التي تضع النصوص والأزرار)، فإن لغة CSS هي "المهندس المعماري" المسؤول عن المظهر الجمالي والتنسيق. هي اللغة التي نستخدمها لتحديد الألوان، الخطوط، والأحجام، وتوزيع العناصر في الصفحة.

○ المهام

في لوحات التحكم (Dashboards) لا يكفي عرض قراءات الحساسات كأرقام مصممة او ازرار تحكم خصوصاً في المواقع الحساسة حيث يفقد المراقب لهذه الواجهة تركيزه مع طول الوقت لذا وجب اضافة ما يلي:

1. التنبيه البصري: تغيير لون النص إلى الأحمر إذا تجاوزت الحرارة حداً معيناً.
2. الوضوح: ترتيب قراءات الحساسات في مربعات (Cards) منظمة يسهل قراءتها.
3. الاستجابة (Responsiveness): جعل واجهة التحكم تظهر بشكل صحيح سواء فُتحت من "هاتف" أو "كمبيوتر".

○ البنية

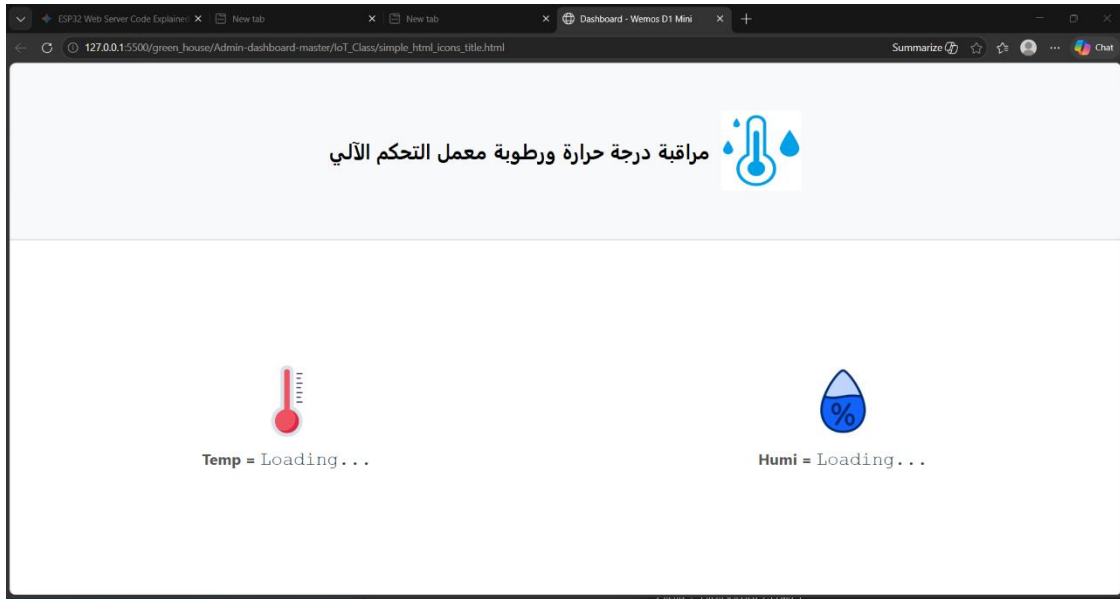
تعتمد اللغة على مبدأ بسيط: "اختر العنصر، ثم حدد الصفة، ثم أعطِ القيمة".

1. المُحدد (Selector): العنصر الذي نريد تنسيقه (مثل h1 أو button).
2. الخاصية (Property): الشيء الذي نريد تغييره (مثل اللون color أو الحجم font-size).
3. القيمة (Value): الدرجة المطلوبة (مثل red أو 20px).

○ طرق إضافة CSS

1. الملف الخارجي (External): كتابة التنسيقات في ملف منفصل ينتهي بـ CSS. وربطه بصفحة الـ HTML.
2. التنسيق الداخلي (Internal): كتابة الأكواد داخل وسم <style> </style> في رأس الصفحة.

سيتم تعديل كود التصميم الخاص بالصفحة لتبدو بالشكل التالي



لو تم مقارنة هذا التصميم بالشكل السابق للصفحة يبدو بوضوح انه تم إضافة بعض المعنى والحيوية الى التصميم فقد تم إضافة بعض الصور التوضيحية للدلالة على الحرارة والرطوبة وكذلك عنوان الصفحة ليبدو واضحاً الهدف من التصميم ومكان اخذ القراءات.

2.2.2 توضيح الجزء البرمجي

○ أولاً CSS

```
<style>
/* 1. إعدادات الصفحة الأساسية لتشغل كامل مساحة الشاشة */
body,
html {
  margin: 0; /* إلغاء الهوامش لضمان عدم وجود فراغات بيضاء حول الصفحة */
  padding: 0;
  height: 100%; /* جعل الصفحة تمتد على كامل طول الشاشة */
  font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
  display: flex; /* لتوزيع العناصر Flexbox تفعيل نظام */
  flex-direction: column; /* ترتيب الصفوف (العنوان ثم البيانات) بشكل رأسي */
}

/* 2. يمثل ثلث الشاشة - تنسيق الصف العلوي (Header) */
.top-row {
  flex: 1; /* تحديد مساحة الثلث من إجمالي مساحة الشاشة */
  background-color: #f8f9fa;
  display: flex;
```

```

/* هي الخاصية التي جعلت الصورة تظهر أولاً من اليمين والنص يتبعها من اليسار */
flex-direction: row-reverse;
justify-content: center; /* توسيط الكتلة (الصورة + النص) في منتصف الصف */
align-items: center; /* التوسيط الرأسي داخل الصف */
gap: 15px; /* المسافة الفاصلة بين يسار الصورة وبداية العنوان */
border-bottom: 2px solid #ddd;
}

.top-row h1 {
  margin: 0; /* تصفير الهوامش التلقائية لتقريب النص من الصورة */
  font-size: 1.8em;
  direction: rtl; /* لضمان تنسيق الجملة العربية بشكل صحيح */
  text-align: right; /* محاذاة النص باتجاه الصورة (جهة اليمين) */
}

.top-row .icon {
  width: 110px;
  height: auto;
  margin: 0; /* إلغاء أي هوامش خارجية للأيقونة */
}

/* يمثل ثلثي الشاشة - (Data Display) تنسيق الصف السفلي 3. */
.bottom-row {
  flex: 2; /* تحديد مساحة الثلثين المتبقية للقراءات */
  background-color: #ffffff;
  display: flex;
  flex-direction: row; /* عرض بطاقات الحساسات بجانب بعضها البعض */
  justify-content: space-around; /* توزيع الحساسات بمسافات متساوية وتلقائية */
  align-items: center;
}

/* 4. (Sensor Card) تنسيق بطاقة الحساس */
.sensor-card {
  text-align: center; /* توسيط النصوص والأرقام تحت الأيقونة */
  padding: 20px;
}

/* تنسيق الأيقونات الخاصة بالحرارة والرطوبة */
.icon {
  width: 100px;
  height: 100px;
  display: block; /* جعل الأيقونة عنصراً مستقلاً ليظهر النص أسفلها */
  margin: 0 auto 15px; /* توسيط الأيقونة أفقياً وترك مسافة تحتها */
}

```

```

/*تنسيق النصوص والقيم الرقمية المستلمة 5.*/
.label-text {
  font-weight: bold;
  font-size: 1.3em;
  color: #555;
}

#temp,
#humi {
  font-size: 1.6em;
  color: #2c3e50; /* لون غامق لتمييز القراءات الرقمية */
  font-family:
    "Courier New", Courier, monospace; /* خطر رقمي يشبه شاشات الآلات */
}
</style>

```

○ ثانيا HTML

```

<div class="top-row">
  
  <h1>مراقبة درجة حرارة ورطوبة معمل التحكم الآلي</h1>
</div>

<div class="bottom-row">
  <div class="sensor-card">
    
    <span class="label-text">Temp = </span>
    <span id="temp">Loading...</span>
  </div>

  <div class="sensor-card">
    
    <span class="label-text">Humi = </span>
    <span id="humi">Loading...</span>
  </div>
</div>

```

○ شرح الكود

- إضافة صورة العنوان temp.humi.png يجب ملاحظة انت تكون الصورة في هذه الحالة في نفس المجلد مع ملف كود الصفحة، الاسم البديل في حالة عدم ظهور الصورة Temperature Icon

```

```

- إضافة صورة العنوان temp.png يجب ملاحظة انت تكون الصورة في هذه الحالة في نفس المجلد مع ملف كود الصفحة، الاسم البديل في حالة عدم ظهور الصورة Temperature

```

```

- إضافة صورة العنوان humi.png يجب ملاحظة انت تكون الصورة في هذه الحالة في نفس المجلد مع ملف كود الصفحة، الاسم البديل في حالة عدم ظهور الصورة Humidity

```

```

- كل هذه الصورة تتبع تنسيق class (icon)

```
/*تنسيق الأيقونات الخاصة بالحرارة والرطوبة*/
.icon {
  width: 100px;
  height: 100px;
  display: block; /* جعل الأيقونة عنصراً مستقلاً ليظهر النص أسفلها */
  margin: 0 auto 15px; /* توسيط الأيقونة أفقياً وترك مسافة تحتها */
}
```

✓ ملاحظات مهمة :

بنهاية هذا الجزء من التجربة وجب التنبيه للنقاط التالية:

1. كود js الحالي لا يعالج مشكلة انقطاع الاتصال مع اللوحة.
 2. كود js الحالي لا يعالج توقف البيانات (الحرارة، الرطوبة) حتى فلا حالة وجود الاتصال مع اللوحة.
- هذه تعتبر ثغرات في النظام يجب معالجتها وسدها، فالأنظمة المدمجة يجب ان تكون مصممة جيداً بحيث تكون قادرة على التعامل مع كل الاحتمالات خصوصاً هند دمجها مع تقنية إنترنت الأشياء.

2.3 التجربة 1 (الجزء الثالث)

في هذا الجزء سنقوم بما يلي:

1. تصوير تصميم الصفحة بإضافة المؤشر (circular progress bar) بحيث يكون لدينا مؤشر خاص بالحرارة وآخر للرطوبة.
2. تطوير كود js للتعامل مع احتمالية انقطاع الاتصال مع اللوحة.
3. تطوير كود js للتعامل مع احتمالية توقف تدفق البيانات مع اللوحة حتى في حالة عدم انقطاع الاتصال.
4. تطوير كود js لعرض نوع الخطأ في حال حدوثه او البيانات المستقبلية في حال عدم حدوث أخطاء.
5. إضافة خاصية عرض التاريخ والوقت الحالي بالصفحة.
6. تطوير كود js ليصبح قادرا على إضفاء طابع حركي تفاعلي على المؤشر.

الشكل التالي يوضح التصميم الجديد لشكل الواجهة



```

<style>
/* --- 1. القواعد العامة والمتغيرات --- */
:root {
  --bg-color: #1a1a2e;
  --card-color: #16213e;
  --accent-color: #0f3460;
  --text-color: #e94560;
  --sub-text: #ffffff;
  --temp-color: #ff4757;
  --humi-color: #2ed573;
  --font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
}

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: var(--font-family);
  background-color: var(--bg-color);
  color: var(--sub-text);
  display: flex;
  flex-direction: column; /* ترتيب العناصر عمودياً لجعل الهيدر في الأعلى */
  height: 100vh;
  overflow: hidden;
}

/* --- 2. الصف العلوي (ممتد ملء الشاشة) --- */
.top-header {
  width: 100%;
  background-color: var(--card-color);
  padding: 1.5rem 0;
  text-align: center;
  border-bottom: 2px solid var(--accent-color);
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.4);
  z-index: 100;
}

```

```

.top-header h1 {
  font-size: 2rem;
  color: var(--sub-text);
  margin-bottom: 0.5rem;
}

.top-header p {
  font-size: 1.2rem;
  color: var(--text-color);
}

/* --- 3. الحاوية الرئيسية (تجمع السايديبار والمحتوى) --- */
.app-container {
  display: flex;
  flex: 1; /* تأخذ باقي ارتفاع الشاشة بعد الهيدر */
  width: 100%;
}

/* القائمة الجانبية */
.sidebar {
  width: 250px;
  background-color: var(--card-color);
  padding: 2rem;
  display: flex;
  flex-direction: column;
  justify-content: space-between;
  border-left: 2px solid var(--accent-color);
}

.sidebar h2 {
  color: var(--text-color);
  font-size: 1.5rem;
  text-align: center;
}

.datetime-container {
  text-align: center;
  background-color: var(--bg-color);
  padding: 1rem;
  border-radius: 10px;
  box-shadow: inset 0 0 10px rgba(0, 0, 0, 0.5);
}

```

```

#time {
  font-size: 1.6rem;
  font-weight: bold;
}
#date {
  font-size: 0.85rem;
  color: #bdc3c7;
  margin-top: 5px;
}

/* منطقة العرض الرئيسية */
.main-content {
  flex: 1;
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 2rem;
}

.dashboard-grid {
  display: flex;
  gap: 4rem;
}

.sensor-card {
  background-color: var(--card-color);
  padding: 2.5rem;
  border-radius: 20px;
  width: 320px;
  height: 320px;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  box-shadow: 0 15px 35px rgba(0, 0, 0, 0.5);
  transition: transform 0.3s ease;
}

.sensor-card:hover {
  transform: translateY(-10px);
}

/* --- CSS باستخدام متغيرات (SVG) الدوائر 4. --- */
.progress-container {
  position: relative;
  width: 160px;
  height: 160px;

```

```

    display: flex;
    align-items: center;
    justify-content: center;
  }

  svg {
    transform: rotate(-90deg);
  }

  .circle-bg {
    fill: none;
    stroke: #2c3e50;
    stroke-width: 15px;
  }

  .circle-progress {
    fill: none;
    stroke-width: 15px;
    stroke-linecap: round;
    stroke-dasharray: 440;
    /* الحساب بناءً على المتغيرات القادمة من السكريبت */
    stroke-dashoffset: calc(440 - (var(--p, 0) / var(--max, 100)) * 440);
    transition: stroke-dashoffset 1s ease-in-out;
  }

  #temp-circle {
    stroke: var(--temp-color);
  }
  #humi-circle {
    stroke: var(--humi-color);
  }

  .progress-value {
    position: absolute;
    font-size: 2.2rem;
    font-weight: bold;
  }

  .sensor-label {
    margin-top: 2rem;
    font-size: 1.6rem;
    font-weight: bold;
    color: #bdc3c7;
  }
}
</style>

```

○ ثانيا كود HTML

```

<div class="top-header">
  <h1>كلية التقنية الإلكترونية-طرابلس</h1>
  <p>مراقبة درجة الحرارة والرطوبة بمعمل التحكم الآلي</p>
</div>

<div class="app-container">
  <div class="sidebar">
    <h2>لوحة التحكم</h2>
    <div class="datetime-container">
      <div id="time">00:00:00</div>
      <div id="date">...جاري التحميل</div>
    </div>
  </div>

  <div class="main-content">
    <div class="dashboard-grid">
      <div class="sensor-card">
        <div class="progress-container">
          <svg width="160" height="160">
            <circle class="circle-bg" cx="80" cy="80" r="70"></circle>
            <circle
              class="circle-progress"
              id="temp-circle"
              cx="80"
              cy="80"
              r="70"
            ></circle>
          </svg>
          <div class="progress-value" id="temp-value">--°C</div>
        </div>
        <div class="sensor-label">الحرارة</div>
      </div>
      <div class="sensor-card">
        <div class="progress-container">
          <svg width="160" height="160">
            <circle class="circle-bg" cx="80" cy="80" r="70"></circle>
            <circle
              class="circle-progress"
              id="humi-circle"
              cx="80"
              cy="80"
              r="70"
            ></circle>
          </svg>

```

```

        </svg>
        <div class="progress-value" id="humi-value">--%</div>
    </div>
    <div class="sensor-label">الرطوبة</div>
</div>
</div>
</div>
</div>

```

○ ثالثا كود js

```

<script>
    function updateTime() {
        const now = new Date();
        document.getElementById("time").textContent =
            now.toLocaleTimeString("ar-EG");
        document.getElementById("date").textContent = now.toLocaleDateString(
            "ar-EG",
            {
                weekday: "long",
                year: "numeric",
                month: "long",
                day: "numeric",
            },
        );
    };
}

async function loadTemp() {
    try {
        const response = await fetch("http://192.168.1.13");
        if (!response.ok) throw new Error();

        const data = await response.json();
        console.log("Data updated: ", data);
        // تحديث النصوص
        document.getElementById("temp-value").textContent = data.temp +
"°C";
        document.getElementById("humi-value").textContent = data.hum + "%";

        // بدون معادلات في السكريبت CSS تحديث المتغيرات في
        const tempCircle = document.getElementById("temp-circle");
        const humiCircle = document.getElementById("humi-circle");

        tempCircle.style.setProperty("--p", data.temp);
        tempCircle.style.setProperty("--max", 50);

        humiCircle.style.setProperty("--p", data.hum);
    }
}

```

```

    humiCircle.style.setProperty("--max", 100);
  } catch (error) {
    console.log("error: ", error);
    document.getElementById("temp-value").textContent = "---";
    document.getElementById("humi-value").textContent = "---";
    document.getElementById("temp-circle").style.setProperty("--p", 0);
    document.getElementById("humi-circle").style.setProperty("--p", 0);
  }
}

setInterval(updateDateTime, 1000);
updateDateTime();
setInterval(loadTemp, 1000);
loadTemp();
</script>

```

○

○ شرح كود js

1. دالة الوقت وهي المسؤولة عن قراءة الوقت واليوم والتاريخ الحالي عبر جهاز الحاسوب الذي يعمل عليه الكود

```

function updateDateTime() {
  const now = new Date();
  document.getElementById("time").textContent =
    now.toLocaleTimeString("ar-EG");
  document.getElementById("date").textContent = now.toLocaleDateString(
    "ar-EG",
    {
      weekday: "long",
      year: "numeric",
      month: "long",
      day: "numeric",
    },
  );
}

```

2. الأوامر الخاصة بتحريك المؤشر رسوميا حسب درجة الحرارة والرطوبة

```

tempCircle.style.setProperty("--p", data.temp);
tempCircle.style.setProperty("--max", 50);

humiCircle.style.setProperty("--p", data.hum);
humiCircle.style.setProperty("--max", 100);

```

- بعد ربط المتغير tempCircle بالعنصر temp-circle في كود HTML

```
const tempCircle = document.getElementById("temp-circle");
```

- استخدم المتغير سابق الذكر في تمرير البيانات

```
document.getElementById("temp-circle").style.setProperty("--p", data.temp);
```

هذا السطر البرمجي يبحث عن الخاصية "p--" في جزء CSS ويغير قيمتها الى data.hum وهي قراءة الحرارة القادمة من الحساس DHT22 المتصلة بلوحة التحكم.

3. تم استخدام try catch لمعالجة احتمالية انقطاع الاتصال

- طالما الاتصال قائم الاسطر البرمجية السابقة ل catch تنفذ مرة كل ثانية.
- عند انقطاع الاتصال تنفذ الأوامر التي تلي سطر catch فيتم تغيير القراءات الى "---" وتصفير المؤشرات.
- يستمر الكود في محاولة الاتصال باللوحة عبر الامر try .
- 4. أوامر عرض القراءات المستقبلية والأخطاء في وضع المطور على المتصفح

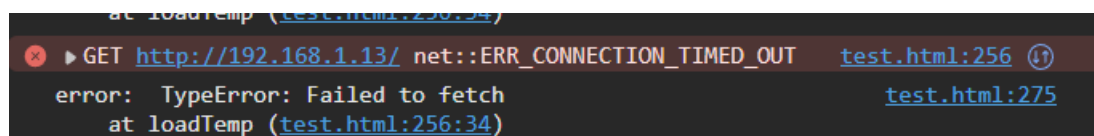
- لعرض البيانات

```
console.log("Data updated: ", data);
```

- لعرض نوع الخطأ

```
console.log("error: ", error);
```

الشكل التالي يوضح رسالة الخطأ و التي ستظهر console عند انقطاع الاتصال باللوحة



مع انتهاء الجزء الخاص ببروتوكول الاتصال HTTP يجب معرفة ان الوصول الى تصميم نظام محكم يدمج الأنظمة المدمجة وتقنيات انترنت الأشياء يحتاج الى الكثير من الوقت والخبرة وازضافة جوانب لا يمكن تعطيها في مذكرة بسيطة مثل هذه، الهدف من هذه المذكرة هو وضع ا لطلب على بداية الطريق.

المراجع العلمية (References)

أولاً: الكتب والمراجع الأساسية (Books & Textbooks)

1. **Monk, S. (2018).** *Programming Arduino: Getting Started with Sketches*. McGraw-Hill Education. [Link to Publisher](#)
2. **Kurose, J. F., & Ross, K. W. (2021).** *Computer Networking: A Top-Down Approach*. Pearson. [Link to Official Site](#)
3. **Margolis, M. (2011).** *Arduino Cookbook*. O'Reilly Media. [Link to O'Reilly](#)

ثانياً: الوثائق التقنية والمصادر عبر الإنترنت (Technical Documentation)

1. **Espressif Systems.** *ESP8266 Datasheet & Hardware Design Guidelines*. [Download PDF Official](#)
2. **Mozilla Developer Network (MDN).** *Using the Fetch API*. [Link to MDN Web Docs](#)
3. **PlatformIO Documentation.** *Embedded Development Environment*. [Link to PlatformIO](#)

ثالثاً: مصادر الذكاء الاصطناعي والتوثيق الرقمي (AI & Digital Repositories)

1. **Google Gemini AI.** [Explore Gemini AI](#).
2. **GitHub Project Repository.** [go to](#)

رابعاً: مراجع الأنظمة المدمجة ولوحة (Embedded Systems & Wemos D1)

1. **Hardware Datasheets** المراجع الرسمية للهاردوير
2. **Espressif Systems.** *ESP8266EX Technical Reference Manual*. [Link to Official PDF](#)
3. **Wemos.cc.** *Wemos D1 R1 Product Page & Pinout Diagram*. [Link to Wemos Wiki](#)

خامساً: مراجع بيئة التطوير والبرمجة (Software Frameworks)

1. **Arduino LLC.** *ESP8266 Core for Arduino Documentation*. [Link to Documentation](#)
2. **Simon, R. (2017).** *IoT Development for ESP8266 Framework*. [Link to Guide](#)

سادسا: هندسة الأنظمة المدمجة (Embedded Engineering Principles)

1. **Noergaard, T. (2012).** *Embedded Systems Architecture: A Comprehensive Guide for Engineers and Programmers*. Newnes. [Link to Reference](#)

سابعا: حساسات قياس البيئة (Environmental Sensors)

2. **Adafruit Industries.** *DHT11, DHT22 and AM2302 Sensors Guide*. [Link to Learning System](#)

قنوات التواصل والدعم الفني(Contact & Support)

لضمان استمرارية التعلم والإجابة على الاستفسارات التقنية المتعلقة بالتجارب البرمجية، يمكنكم التواصل عبر الوسائل التالية:

• البريد الإلكتروني rabei.shwehdi@gmail.com

• قناة التليجرام (للتحديثات السريعة) :



• مستودع الأكواد (GitHub) : [go to](#)